



UNIVERSIDADE ESTADUAL PAULISTA
“JÚLIO DE MESQUITA FILHO”

INSTITUTO DE CIÊNCIA E TECNOLOGIA
UNIVERSIDADE ESTADUAL PAULISTA

RELATÓRIO ESTRUTURA DE DADOS

SISTEMA DE AVALIAÇÃO DE DESEMPENHO
EM ESTRUTURAS DE DADOS: TELEMETRIA
AUTOMOTIVA

Professor:

Prof. Dr. Leopoldo André
Dutra Lusquino Filho

Aluno - RA:

George Lucas Lazarim - 251270475

29 de junho de 2026

UNIVERSIDADE ESTADUAL PAULISTA - UNESP
INSTITUTO DE CIÊNCIA E TECNOLOGIA DE SOROCABA
CURSO DE ENGENHARIA DE CONTROLE E AUTOMAÇÃO

George Lucas Lazarim

**SISTEMA DE AVALIAÇÃO DE DESEMPENHO EM ESTRUTURAS
DE DADOS: TELEMETRIA AUTOMOTIVA**

Orientador: Prof. Dr. Leopoldo André
Dutra Lusquino Filho

29 de junho de 2026

Resumo

Este trabalho avalia o desempenho computacional de estruturas de dados aplicadas a um sistema de diagnóstico de telemetria automotiva, simulando as restrições físicas de uma Unidade de Controle Eletrônico (ECU). Para a validação, implementou-se um gerador de dados sintéticos baseado no comportamento de um motor turbo, que adiciona ruído por meio da transformada de Box-Muller e injeta falhas mecânicas de forma probabilística. As estruturas implementadas, que são a Fila Estática Circular, a Tabela Hash, o Bloom Filter, a Max-Heap e a Skip List, foram submetidas a *benchmarks* de estresse com até 100.000 pacotes. Os resultados mostram que a eliminação da alocação dinâmica em *buffers* contíguos reduz de forma expressiva o tempo das operações e diminui o risco de fragmentação de memória. A análise comprovou que a dispersão da Tabela Hash e as verificações do Bloom Filter mantêm eficiência constante apenas quando há memória sobressalente, perdendo desempenho sob saturação. Além disso, os testes revelaram o peso da latência de memória e das falhas de *cache* (*cache misses*) na remoção da raiz da Max-Heap sob alta carga. Observou-se, por fim, que o maior consumo de memória da Skip List, causado pela alocação de ponteiros, é o preço necessário para garantir buscas rápidas por tempo no histórico de falhas. Conclui-se que o dimensionamento correto e a escolha das estruturas determinam a capacidade de um sistema embarcado de processar alertas críticos em tempo real sem ultrapassar os limites do *hardware*.

Palavras-chave: Telemetria Automotiva. Estruturas de Dados. Sistemas Embarcados. Benchmark de Desempenho. Diagnóstico de Falhas.

Sumário

1	Introdução	3
2	Dataset e Geração Sintética	3
3	Cenários de Restrição	4
4	Operações Adicionais	5
5	Visualização	6
6	Estruturas Implementadas	8
6.1	Fila	8
6.1.1	Fila Dinâmica	8
6.1.2	Fila Estática Circular (Estrutura Otimizada)	9
6.2	Tabela Hash	9
6.3	Bloom Filter	10
6.4	Max-Heap	10
6.5	Skip List	11
7	Metodologia de Benchmark	12
8	Resultados e Discussão	14
8.1	Fila Dinâmica e Fila Estática Circular (Estrutura Otimizada)	14
8.2	Hash	18
8.3	Bloom Filter	22
8.4	Max-Heap	26
8.5	Skip List	28
9	Conclusão	31
10	Apêndice	31
11	Bibliografia	32

1 Introdução

A Unidade de Controle Eletrônico (ECU) é o núcleo de processamento dos veículos modernos. Nela, o gerenciamento contínuo de dados de telemetria, como rotação do motor, pressão do turbo e temperatura, exige processamento em tempo real para garantir a integridade mecânica. Como isso ocorre sob forte restrição de hardware, com memória RAM escassa e processamento limitado a um único núcleo, a eficiência algorítmica torna-se um requisito de confiabilidade. Um atraso no diagnóstico de uma falha de lubrificação ou de uma sobrepressão no turbo pode causar danos graves.

O objetivo deste trabalho é implementar e avaliar o desempenho de diferentes estruturas de dados operando de forma integrada em um simulador de telemetria automotiva. A proposta não é criar um sistema de controle de injeção, mas usar a modelagem de um motor Volkswagen AP 1.6 8V turbo como ambiente de estresse para testar os limites das estruturas. O sistema atua como ferramenta de diagnóstico, recebendo um fluxo sintético e ininterrupto de dados, identificando anomalias e priorizando alertas de forma autônoma.

A arquitetura emprega cinco estruturas, cada uma com um papel definido. A Fila Estática Circular absorve o fluxo bruto de recepção. A Tabela Hash traduz as falhas. O Bloom Filter evita repetir alertas já emitidos. A Max-Heap define a prioridade de tratamento, e a Skip List indexa o histórico ordenado da telemetria. A validação submete cada estrutura a *benchmarks*, variando seus parâmetros internos e medindo o efeito no tempo de execução e no consumo de memória.

2 Dataset e Geração Sintética

A geração sintética do dataset foi escolhida para ter controle total sobre os cenários de teste. Bases públicas de telemetria para o motor Volkswagen AP 1.6 8V são escassas e operam quase sempre em regime normal. Já o modelo sintético permite fixar a semente do gerador de números pseudoaleatórios, o que garante a reprodutibilidade, e ainda forçar a ocorrência de falhas de forma controlada.

Para manter a fidelidade física, cada amostra é uma representação do motor, e não um número solto. A cada ciclo de 12 ms, o pacote registra seis grandezas interdependentes. O tempo, em milissegundos, define a ordenação cronológica que fundamenta as buscas. A posição da borboleta (TPS) é a variável de demanda e oscila de 0 a 100%. Em resposta, a rotação opera entre 900 RPM na marcha lenta e 7.200 RPM no corte. A pressão do turbo varia do vácuo até 1,5 bar, a do óleo flutua até o limite de 5,0 bar da válvula de alívio, e a temperatura parte de 30 graus, podendo passar de 105 graus Celsius sob condições severas. As grandezas também

reagem umas às outras. A rotação cresce com o TPS e recebe um multiplicador da pressão do turbo, enquanto a temperatura sobe com picos prolongados de RPM ou quedas na pressão do óleo.

Para simular sensores analógicos reais e evitar curvas artificiais, aplicou-se ruído sobre as leituras com a transformada de Box-Muller, que gera ruído de distribuição Gaussiana a partir de variáveis uniformes. O ruído foi aplicado à marcha lenta, à pressão do óleo e à leitura do turbo, que são as grandezas mais sujeitas a oscilação mecânica.

O modelo também injeta falhas. Um acumulador de fadiga cresce sempre que o motor opera fora da faixa ideal, como em rotação extrema ou temperatura alta. Ao passar dos limiares de estresse, o sistema injeta falhas de forma probabilística, derrubando a pressão do óleo ou cortando a eficiência do turbo. É esse mecanismo que inunda o fluxo com anomalias e fornece o material de estresse para o filtro e para a fila de prioridades.

Por fim, o volume de dados foi dimensionado conforme cada teste. No laço principal do sistema integrado, a arquitetura processa um fluxo de 20.000 pacotes de telemetria. Já os *benchmarks* isolados escalam bem acima desse valor, chegando a 60.000 elementos na avaliação do dicionário de falhas e a 100.000 pacotes nos testes de histórico da Skip List. Esse volume maior nos *benchmarks* foi necessário porque cargas pequenas seriam processadas inteiras na cache do processador, o que esconderia o custo real de latência de memória de estruturas como a Max-Heap. Escalar para centenas de milhares de amostras permite saturar o fator de carga e revelar as diferenças de desempenho entre as variantes.

3 Cenários de Restrição

O sistema foi avaliado sob condições restritivas que simulam as limitações de uma Unidade de Controle Eletrônico real. Foram aplicados cinco cenários, um de cada categoria exigida pelo projeto.

Na categoria de Memória, adotou-se o cenário R1, que impõe um teto de memória RAM para a execução. O ambiente de testes foi travado limitando a memória virtual do processo a 128 megabytes. Essa restrição orientou a escolha de estruturas compactas, como a Fila Estática Circular e a árvore implícita da Max-Heap, que dispensam ponteiros em excesso. O monitoramento de recursos registrou um pico de consumo de apenas 4 megabytes, o que mostra que o sistema opera com folga.

Na categoria de Processamento, o sistema integrado foi submetido à restrição

R6, que limita a execução a um único núcleo. A condição foi garantida com o comando `taskset`, que elimina a migração entre núcleos e expõe o custo de cada algoritmo. O sistema foi projetado para essa limitação, usando operações de tempo constante na Tabela Hash e no buffer de recepção para evitar a perda de pacotes sob alta frequência.

Na categoria de Latência, o projeto operou sob a restrição R14, com uma fila única de eventos e sem paralelismo. O laço de consumo foi escrito de forma sequencial, sem dividir a carga em várias threads. A arquitetura concentra o tratamento das falhas na Max-Heap, o que garante que o alerta de maior severidade seja resolvido em tempo logarítmico sem depender de execução concorrente.

Na categoria de Dados, a modelagem incorporou a restrição R20, que força o sistema a trabalhar com dados sintéticos de baixa confiabilidade. Esse cenário foi implementado no gerador de telemetria por meio da transformada de Box-Muller e do acumulador de fadiga, que injetam imprecisões de distribuição Gaussiana e anomalias abruptas, como a falha do turbo, sobre as leituras dos sensores. Com isso, o Bloom Filter e o dicionário de falhas foram submetidos a uma carga contínua de anomalias, mantendo a eficiência de tradução e filtragem mesmo quando a fonte dos dados é imprecisa.

Por fim, a categoria Algorítmica e Estrutural foi coberta pela restrição R25, que exige lidar com grandes volumes de entrada contínua e indexação constante. O cenário foi estabelecido no laço principal, com a ingestão ininterrupta de 20.000 pacotes de telemetria. Para suportar essa demanda, o sistema faz a alocação incremental de nós, a ordenação de prioridades e o remapeamento dos atalhos de forma dinâmica à medida que as leituras chegam. Assim, a fila de falhas e o histórico ficam sempre ordenados e prontos para consulta.

4 Operações Adicionais

Além das operações básicas de inserção, busca e remoção, o sistema implementa operações de cálculo estatístico sobre os dados brutos dos sensores. Elas foram integradas ao buffer de recepção e funcionam de forma idêntica nas duas variantes da Fila, a estática e a dinâmica.

A primeira é a média aritmética da rotação do motor. Uma função percorre os pacotes armazenados no buffer e calcula a média do RPM na janela de dados atual. Essa média ajuda a determinar o regime real de operação do motor, filtrando pequenas oscilações dos sensores e dando uma base estável para o diagnóstico.

A segunda é a taxa média de variação da temperatura do arrefecimento. Ao

comparar a amostra mais antiga com a mais recente, a função calcula o salto térmico pelo intervalo de tempo decorrido. Essa operação por diferenças finitas permite identificar tendências de aquecimento antes que a temperatura atinja o limite crítico da junta do cabeçote, o que viabiliza alertas preventivos.

A terceira é o pico de pressão do turbo. A função percorre a janela de dados ativa e isola o maior valor de sobrealimentação atingido. Diferente das duas anteriores, que suavizam o ruído para revelar uma tendência, essa busca captura um extremo pontual. Em um motor turboalimentado, um único pico de sobrepressão já configura risco para os pistões, então a rotina garante o registro desse valor, por mais rápido e isolado que seja o evento.

5 Visualização

Para permitir a exploração do comportamento das estruturas, o sistema conta com dois módulos de visualização. O primeiro é uma interface interativa no terminal, que não exibe apenas os dados brutos dos sensores, mas mostra o estado das estruturas operando em conjunto.

Durante a execução, o usuário acompanha um *pipeline* dinâmico. O fluxo de telemetria entra pelo buffer de recepção e passa pela checagem de limites, que gera um código de erro bruto ao detectar uma anomalia. Esse código é enviado à Tabela Hash para ser traduzido. Em seguida, o alerta passa pelo Bloom Filter, que verifica se a falha já foi registrada no ciclo atual. Sendo um diagnóstico novo, ele é armazenado no histórico da Skip List e priorizado na Max-Heap.

O painel interativo, mostrado na Fig. 1, faz essa priorização ao esvaziar a Max-Heap e imprimir os diagnósticos na ordem ditada pela gravidade da anomalia. As cores ajudam na leitura, com os alertas de severidade extrema em vermelho e as previsões de falha por tendência em magenta.

```
gll@asusf15ubuntu:~/Documents/esd/data-structure-project$ ( ulimit -v 131072 ; taskset -c 0 ./main )

=====
ECU - Telemetria AP 1.6 8v Turbo
=====
Restricao R5 ativa: historico limitado a 5000 pacotes (descarte dos antigos).
processamento concluido (20000 amostras).

=== RESUMO DA EXECUCAO ===
Historico (skip list): 5000 pacotes
Hash: 33 entradas | maior cadeia: 7 | colisoes: 21
Bloom: ocupacao 0.018 | FP estimado 0.00001

=== PAINEL DE ALERTAS (heap, por severidade) ===
PREVISAO [sev 4] PR524 Perda de pressao iminente (prevista)
ALERTA [sev 4] P0299 Falha de turbina (sem pressao)
ALERTA [sev 2] P0219 Rotacao excessiva (overrev)

(grafico salvo em dashboard.png)

=== CONSULTA DE HISTORICO (skip list) ===
Digite um tempo em ms (multiplos de 12, de 12 a 240000) ou 'q' para sair.
tempo> 240000
t=240000 ms -> RPM=2311 | TPS=2% | oleo=1.56 bar | temp=89.8 C | boost=-0.15 bar
tempo> 200000
nada em t=200000 (multiplos de 12; sob R5 os antigos sao descartados)
tempo> 29580
nada em t=29580 (multiplos de 12; sob R5 os antigos sao descartados)
tempo> 204000
t=204000 ms -> RPM=1567 | TPS=0% | oleo=1.09 bar | temp=89.8 C | boost=-0.59 bar
tempo> q

encerrando.

=== MEMORIA ===
Pico de memoria residente (RSS): 2016 KB (~2.75 MB)
(teto da R1: 131072 KB = 128 MB)
gll@asusf15ubuntu:~/Documents/esd/data-structure-project$
```

Figura 1: Painel interativo em terminal exibindo o fluxo de telemetria.

Para cada alerta, o sistema mostra o código OBD2 e faz uma busca na Skip List para recuperar a condição exata do motor (RPM, temperatura e pressões de óleo e turbo) no milissegundo da detecção. A interface ainda oferece uma consulta de histórico, na qual o usuário digita um instante de tempo e o sistema varre as camadas da Skip List para retornar à leitura daquele momento.

A segunda camada de visualização é gráfica. O histórico de telemetria é exportado da estruturas em memória para um arquivo de valores separados por vírgula (`telemetry.csv`). Durante o desenvolvimento, processar esses dados direto no Gnu-plot estourou o teto de 128 megabytes de RAM imposto pela restrição R1. Para não violar esse limite, a etapa de renderização foi separada do sistema.

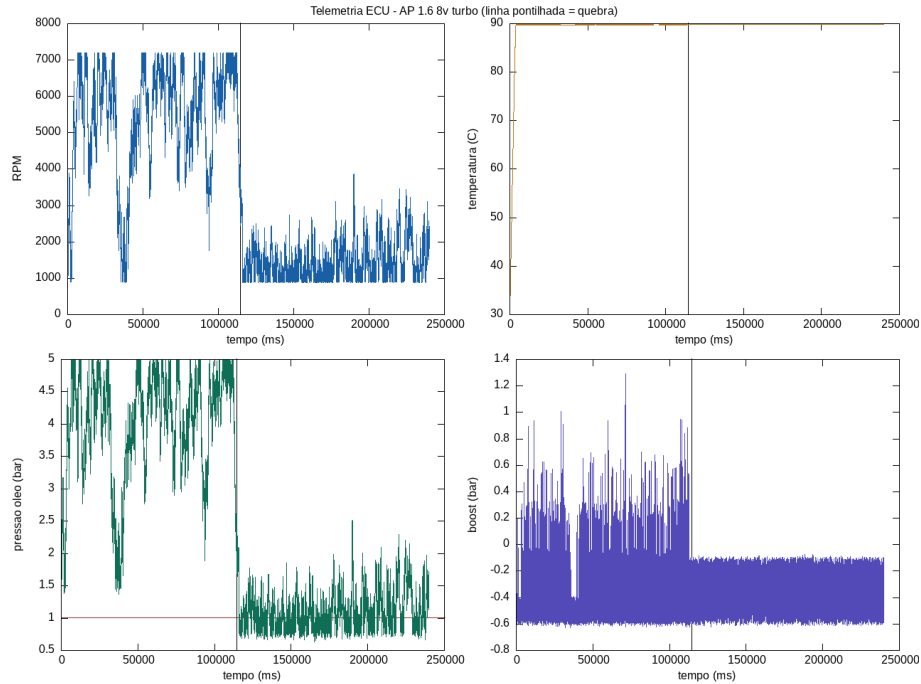


Figura 2: Dashboard gerado a partir do histórico exportado da Skip List, relacionando o comportamento das grandezas críticas do motor aos limites físicos e instantes de falha

A visualização final é uma imagem estática em formato `.png`, mostrada na Fig. 2, gerada fora do sistema a partir dos dados exportados. Esse *dashboard* exibe as quatro grandezas críticas do motor ao mesmo tempo. A plotagem sobrepõe os dados sintéticos às linhas de limite físico, como a fronteira de 105 graus Celsius e o limiar de 1,0 bar de pressão positiva, marcando o instante exato da injeção das anomalias.

6 Estruturas Implementadas

6.1 Fila

6.1.1 Fila Dinâmica

A Fila Dinâmica é uma estrutura *FIFO* (First In, First Out) sem tamanho máximo definido. Ela cresce e diminui sob demanda, usando alocação dinâmica de memória com `malloc` e `free`. Cada dado fica em um nó, gerido por uma estrutura de controle.

A estrutura tem duas partes, o nó e a estrutura de controle. O nó armazena

o dado e o ponteiro para o próximo nó. Sozinho ele não forma uma Fila, então a estrutura de controle reúne os nós com dois ponteiros, um para o início e outro para o fim.

Suas operações principais são enfileirar (enqueue) e desenfileirar (dequeue), ambas com complexidade $O(1)$, como será demonstrado na seção 8. Elas atuam direto nas extremidades da estrutura, sem percorrer ou reorganizar os outros elementos.

6.1.2 Fila Estática Circular (Estrutura Otimizada)

A Fila Estática Circular é o buffer de recepção, responsável pela entrada de dados em alta frequência da ECU. Ela ficou nessa posição porque o fluxo de dados é variável e nem sempre o consumo acompanha a entrada. Sem um buffer adequado, parte dos dados seria perdida ou o sistema ficaria lento.

Ela também é uma estrutura *FIFO*, mas usa um vetor de tamanho fixo já alocado na memória. Sua principal característica é o reaproveitamento dos espaços vazios. Quando o índice chega ao fim do vetor, ele volta ao início, desde que haja espaço livre.

A diferença principal para a Fila Dinâmica está nos ponteiros. A Fila Estática Circular não precisa deles para ligar os elementos, pois o vetor já foi alocado em memória contígua, então basta a aritmética de índices para inserir e remover. Por isso a estrutura de controle guarda apenas o índice de início, o de fim, o tamanho e o vetor. Suas operações também são $O(1)$ e mais rápidas que as da Fila Dinâmica, já que não pagam alocação dinâmica por operação e o armazenamento contíguo favorece a localidade de cache.

6.2 Tabela Hash

A Tabela Hash é o módulo de tradução e indexação, com número de *buckets* fixo. Ela funciona como o dicionário de falhas do sistema, traduzindo em tempo médio constante os códigos numéricos brutos de erro em códigos padronizados no formato *OBD2* (Diagnostic Trouble Code, DTC).

A estrutura mapeia uma chave a uma posição do vetor por meio de uma função matemática (hashing), o que permite busca e remoção médias em $O(1 + \alpha)$. O termo α é o fator de carga, diretamente proporcional ao número de elementos e inversamente proporcional ao tamanho da tabela, conforme a Eq. 1.

$$\alpha = \frac{n}{\text{tamanho}} \quad (1)$$

Quanto maior o fator de carga, maior a chance de colisões, que precisam ser tratadas. Para a finalidade adotada, com n pequeno e limitado, o tratamento por encadeamento é válido e tem risco quase nulo de degradar a complexidade para $O(n)$. Nesse tratamento, cada índice aponta para uma lista, e a cada colisão o novo elemento entra no topo dela.

A função hash que define o índice de cada chave é o módulo da chave pelo tamanho da tabela. O uso de um número primo para esse tamanho ajuda a distribuir as chaves e reduz as colisões.

6.3 Bloom Filter

O Bloom Filter é o filtro da Max-Heap. Quando uma falha acontece, os sensores continuam enviando dados fora do limite de funcionamento. Sem um filtro, falhas que já estavam sendo tratadas continuariam inundando a Max-Heap.

É uma estrutura estática e probabilística, formada por um vetor de bits e k funções hash. Para inserir um elemento, passa-se a chave pelas k funções e ligam-se os bits das posições correspondentes, com complexidade $O(k) = O(1)$. Para verificar se um elemento já passou pelo filtro, faz-se o mesmo processo, mas em vez de ligar os bits, checa-se se eles estão ligados, com a mesma complexidade.

O filtro guarda apenas marcas, ou seja, os bits, e não os elementos. Por isso não é possível recuperar o que entrou nem afirmar com certeza que uma chave está presente. Isso faz com que ele possa gerar um falso positivo, mas nunca um falso negativo.

A estrutura não suporta remoção. Como vários elementos podem compartilhar o mesmo bit, zerar um bit para remover uma chave apagaria a marca de todas as outras chaves que dependem dele.

6.4 Max-Heap

A Max-Heap é onde as falhas d motor são geridas por prioridade de severidade. Neste projeto, o sistema atua só como ferramenta de diagnóstico e alerta, sem interferir no controle do motor, ou seja, ele não corta a ignição nem reduz o RPM. Ele apenas reporta ao usuário os problemas possíveis ou já ocorridos em ordem estrita de prioridade.

Baseada em árvore binária, a Max-Heap é formada por pais e filhos. O preenchimento vai da esquerda para a direita e todo pai é maior ou igual aos filhos, propriedade que permite implementá-la em um vetor, sem ponteiros. Os índices de pai e filho são calculados a partir do índice i do elemento atual.

$$i_{pai} = \left\lfloor \frac{i-1}{2} \right\rfloor \quad (2)$$

$$i_{filhoE} = 2i + 1 \quad (3)$$

$$i_{filhoD} = 2i + 2 \quad (4)$$

O elemento entra no fim do vetor, na posição tamanho $- 1$, e sobe na árvore comparando-se com o pai (Eq. 2). Quando é maior que o pai, os dois trocam de posição, e isso se repete até ele encontrar o lugar certo. Por conta da altura da árvore, a inserção tem complexidade $O(\log n)$.

A raiz, que é o máximo, é removida. O último elemento do vetor assume o topo e desce, trocando de posição com o maior dos filhos enquanto for menor que ele, até reencontrar a posição válida. Como no pior caso ele percorre da raiz até uma folha, a remoção também é $O(\log n)$.

A Max-Heap não foi projetada para busca arbitrária. Encontrar um elemento que não seja o máximo exige, no pior caso, varrer todo o vetor, com complexidade $O(n)$, pois a propriedade de heap ordena apenas a relação entre pai e filho, e não a árvore inteira. Por isso a métrica de tempo de busca não é reportada para esta estrutura.

6.5 Skip List

A Skip List armazena o histórico dos sensores na ECU. Ela indexa os dados por tempo, o que permite busca rápida por instante e também busca por faixa, já que mantém os elementos ordenados.

A estrutura é uma lista ordenada com vários níveis sobrepostos. O nível mais baixo contém todos os elementos, e cada nível acima funciona como um atalho que pula vários elementos de uma vez. A busca começa no nível mais alto e avança enquanto o próximo elemento for menor que o procurado. Quando passa do alvo, desce um nível e continua, refinando a cada descida até chegar ao elemento no nível base.

O que mantém a velocidade é a forma de inserção. A altura de cada nó, ou seja, em quantos níveis de atalho ele participa, é sorteada quando o dado entra. Isso distribui as alturas de modo que os atalhos fiquem eficientes em média, sem precisar de reorganização.

Como busca, inserção e remoção dependem de localizar uma posição por esses atalhos, as três operações têm complexidade $O(\log n)$ em média. A escolha pela Skip List, em vez de uma árvore balanceada, se deu para eliminar o custo das rotações de nós. Em um cenário onde o motor gera dados massivos e em alta frequência o tempo todo, parar para rebalancear uma árvore deixaria o sistema lento.

7 Metodologia de Benchmark

A metodologia de benchmark adotada foi comparar variantes da mesma estrutura sob um parâmetro. Como cada estrutura tem uma finalidade específica no sistema, a comparação direta entre elas seria metodologicamente falha. A Skip List não deve ser comparada com o Bloom Filter, e o mesmo vale para as demais.

Para comparar variantes de uma estrutura, escolheu-se um parâmetro de arquitetura que faz sentido para cada caso. Assim, cada estrutura passou por testes individuais, focados em suas próprias configurações. O objetivo foi observar como essas escolhas afetam o tempo de execução e o consumo de memória, sempre escalando o volume de dados para ver o comportamento sob estresse.

Para as Filas, o parâmetro foi a forma de alocação de memória. A Fila Estática Circular e a Fila Dinâmica foram medidas lado a lado nos tempos de enfileirar e desenfileirar, para confirmar na prática o peso da alocação dinâmica no buffer de recepção.

Para a Tabela Hash, o parâmetro foi o tamanho inicial do vetor. Foram testadas tabelas com capacidades diferentes, sempre números primos, para entender como a falta ou a sobra de espaço afeta as colisões, o fator de carga e o tamanho da maior cadeia, fatores que atrasam a busca.

Para o Bloom Filter, o foco foi a quantidade de funções hash por elemento. Variar esse número mostra a troca clássica da estrutura, que é gastar mais tempo nas operações de inserção e verificação em troca de uma taxa menor de falsos positivos.

Para a Max-Heap, foi feito apenas o teste de estresse, pois ela não tem parâmetro para variar. É uma árvore binária implícita, sem grau de liberdade de configuração. O Bloom Filter também tem complexidade fixa, mas, ao variar o número de funções hash, a taxa de falso positivo muda.

Para a Skip List, o parâmetro foi a probabilidade de um nó subir de nível. Uma probabilidade maior gera mais atalhos, o que acelera a busca à medida que a lista cresce. Essa otimização tem um custo, já que exige mais memória e eleva os tempos de inserção e remoção, por causa do número maior de ponteiros a gerenciar.

Para garantir a confiabilidade dos resultados, os benchmarks seguiram uma rotina padronizada. O primeiro passo foi sempre gerar toda a massa de dados sintéticos da telemetria de forma controlada. Com os pacotes prontos, o sistema não avaliou o desempenho por uma única execução, que poderia ser corrompida por flutuações do processador.

Cada variante recebeu o mesmo dataset, e a mesma operação foi executada várias vezes seguidas. Ao fim de cada ciclo, o algoritmo ordenava os tempos e extraía a mediana, eliminando a influência de *outliers*. Foi esse valor que serviu para construir os gráficos da seção 8.

Além de manter o dataset de entrada idêntico, foi preciso isolar a geração de números aleatórios durante os testes. Como a Skip List consome a função `rand()` para sortear a altura dos nós, se o benchmark usasse o mesmo gerador para escolher o que buscar ou apagar, as variantes consumiriam a sequência em ritmos diferentes.

Para medir tempos na casa dos nanossegundos com precisão, o ambiente usou três controles em conjunto. O primeiro foi a medição em lote, pois cronometrar uma operação por vez faria o peso da chamada do relógio dominar o tempo medido. Em vez disso, o sistema agrupava milhares de operações em um bloco e dividia o tempo total pelo número de operações. O segundo foi a fixação de núcleo, com os testes travados sempre no mesmo núcleo, o que reduz a variação causada pela migração entre núcleos. O terceiro foi o uso do relógio monotônico, que garante uma contagem linear do tempo.

8 Resultados e Discussão

8.1 Fila Dinâmica e Fila Estática Circular (Estrutura Otimizada)

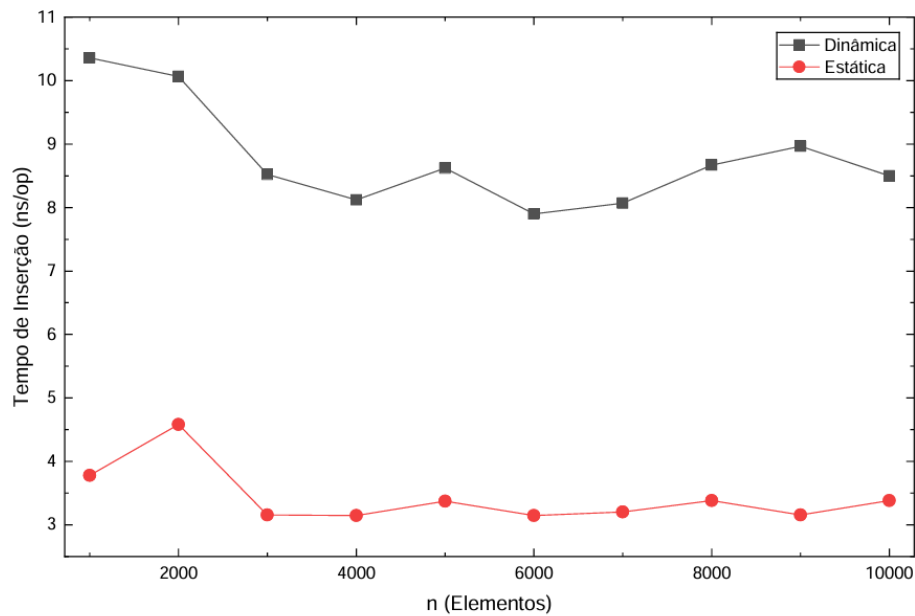


Figura 3: Comparativo do tempo de inserção entre a Fila Dinâmica e a Fila Estática Circular sob estresse de dados.

A comparação direta entre a Fila Dinâmica e a Fila Estática Circular revela o custo da alocação de memória. Na Fig. 3, a diferença aparece logo de início. A Fila Dinâmica, por chamar `malloc` a cada novo pacote, tem um custo extra e opera na faixa de 8 a 11 nanossegundos por operação.

Já a Fila Estática Circular, que trabalha sobre um vetor já alocado e usa apenas aritmética de índices para avançar, registra tempos baixos e constantes, entre 3 e 4 nanossegundos por operação, sem depender da carga de elementos.

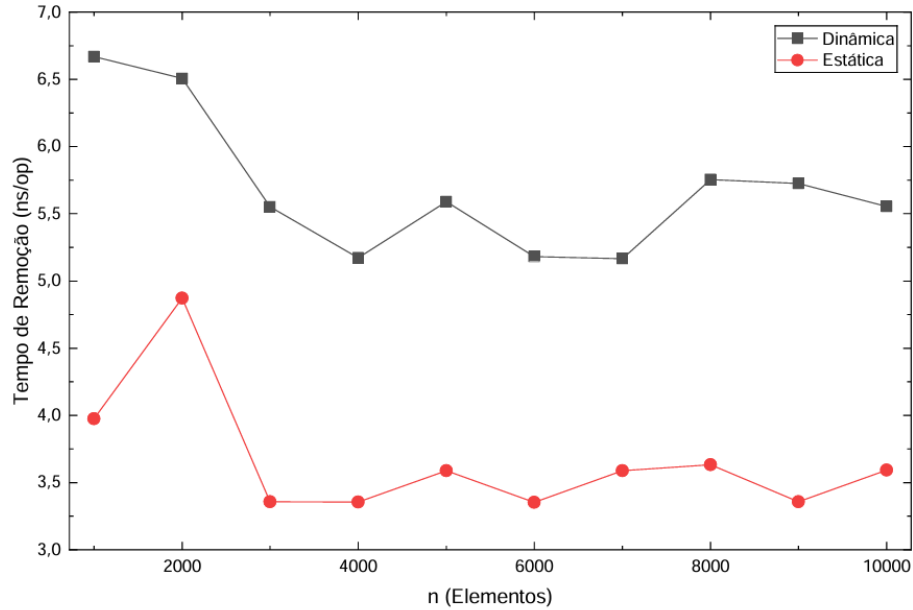


Figura 4: Tempo de remoção do buffer de recepção nas implementações estática e dinâmica.

A mesma diferença se repete na Fig. 4. Desenfileirar na estrutura dinâmica cobra o custo da função `free` para devolver o espaço do nó à memória, o que leva a operação para a faixa de 5 a 7 nanossegundos. A Fila Estática Circular apenas avança o índice de início, mantendo a remoção entre 3 e 4 nanossegundos por operação.

Esses resultados confirmam que, embora as duas estruturas tenham complexidade teórica $O(1)$, as funções `malloc` e `free` têm peso e aumentam o tempo das operações.

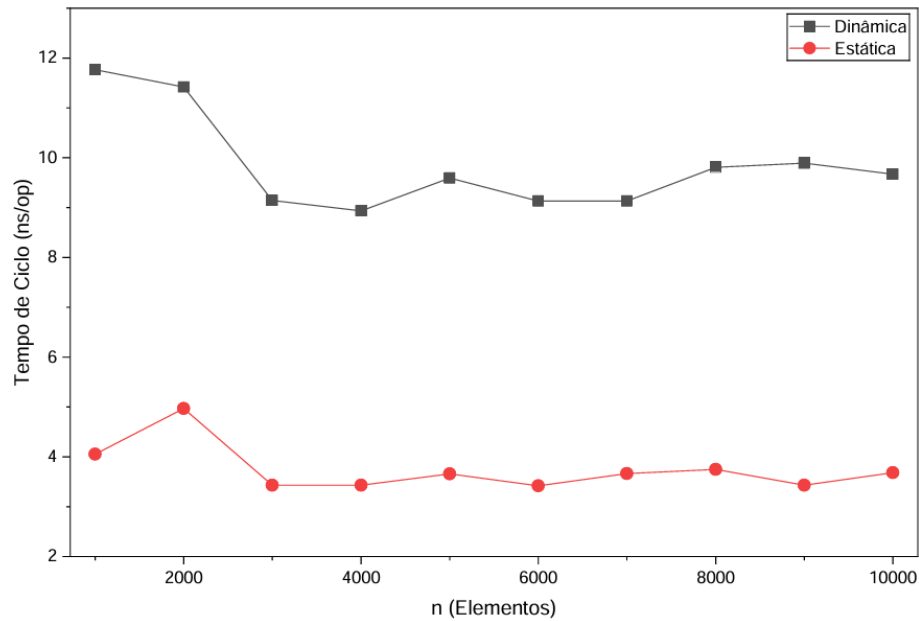


Figura 5: Tempo de ciclo completo (inserção e remoção), destacando o gargalo de alocação da Fila Dinâmica.

Para consolidar a análise, a Fig. 5 mostra o tempo de um ciclo completo de enfileiramento e desenfileiramento. Essa é a métrica que melhor reflete a operação contínua da ECU, onde os dados entram e saem do buffer o tempo todo. Como esperado, o tempo de ciclo soma a diferença das análises individuais.

A Fila Dinâmica, penalizada pela dupla chamada das funções de alocação, leva de 9 a 12 nanossegundos por ciclo. A Fila Estática Circular, livre dessas funções, faz o mesmo trabalho em 3,5 a 5 nanossegundos por ciclo. Na prática, isso significa processar o fluxo contínuo na metade do tempo, o que comprova que a alocação prévia contígua é essencial para a recepção dos sensores não virar o gargalo do sistema.

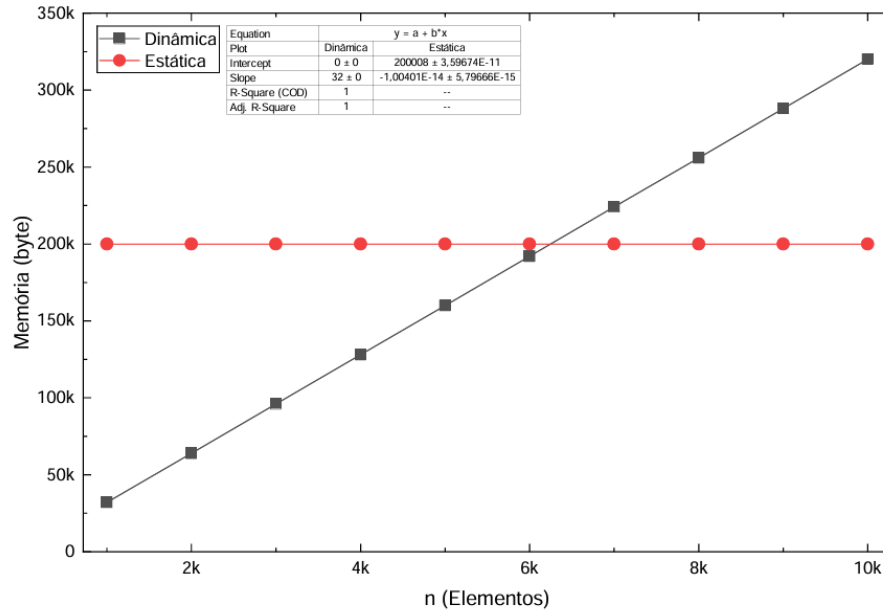


Figura 6: Consumo de memória das Filas: alocação dinâmica sob demanda versus reserva estática contígua.

Para fechar a análise do buffer de recepção, a Fig. 6 mostra a contrapartida dessa otimização de tempo. A Fila Dinâmica cresce de forma linear, com cada novo pacote exigindo exatos 32 bytes, o que resulta na reta $y = 32x$. À primeira vista essa abordagem parece eficiente, pois não gasta memória com o buffer vazio.

A Fila Estática Circular adota uma estratégia inflexível e trava um bloco fixo e contíguo de cerca de 200 Kbytes logo na inicialização, suficiente para sua capacidade máxima de 10.000 elementos, e mantém esse consumo constante.

Embora reservar todo esse espaço antes pareça um desperdício quando o fluxo está baixo, essa decisão é importante para a estabilidade da ECU. Com um custo de memória fixo, a estrutura reduz a fragmentação da RAM que a alocação dinâmica repetida causaria ao longo de horas de uso. Além disso, a reserva prévia protege o sistema contra falhas por falta de memória, que poderiam ocorrer se o processador tentasse alocar milhares de nós ao mesmo tempo durante um pico de alta frequência. Por isso, gastar 200 Kbytes fixos é um preço pequeno para garantir a estabilidade do fluxo contínuo de telemetria.

8.2 Hash

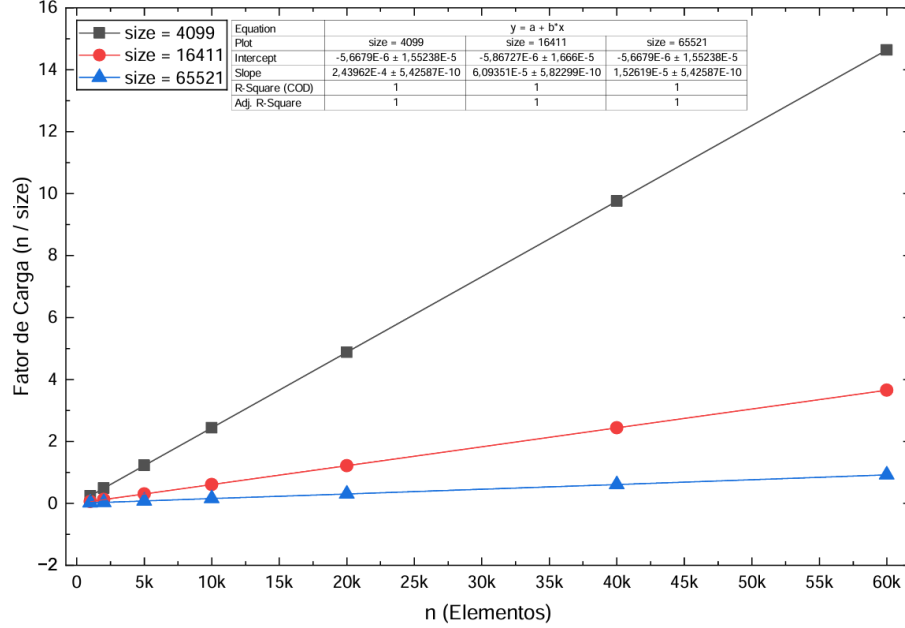


Figura 7: Comportamento do fator de carga da Tabela Hash ao longo da entrada contínua de dados.

Como mostra a Fig. 7, o fator de carga (Eq. 1) da Tabela Hash cresce de forma linear conforme novos pacotes são inseridos. Como a arquitetura não faz redimensionamento dinâmico, o tamanho do vetor base fica travado enquanto o volume de dados aumenta.

O gráfico deixa claro o impacto do subdimensionamento. A variante menor, com apenas 4099 posições, satura rápido. Ela passa do limiar ideal de $\alpha = 1,0$ antes de chegar a 5.000 elementos, o que significa empilhar várias colisões na mesma lista encadeada e degradar a busca.

Já a variante com 65.521 posições absorve a carga com folga, mantendo o fator de carga abaixo de 1,0 durante quase todo o estresse. É essa sobra de memória que deixa a função de dispersão trabalhar livremente, mantendo os encadeamentos curtos e as operações em $O(1 + \alpha) \mid \alpha \approx 0$.

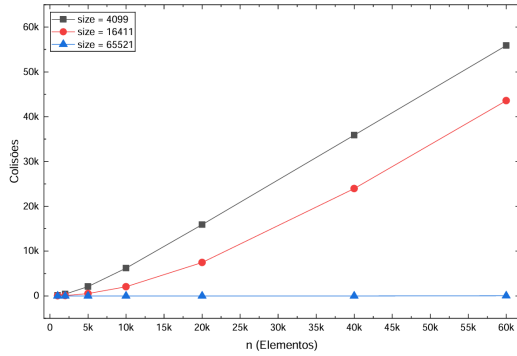


Figura 8: Volume absoluto de colisões registradas nas três variantes de capacidade da tabela.

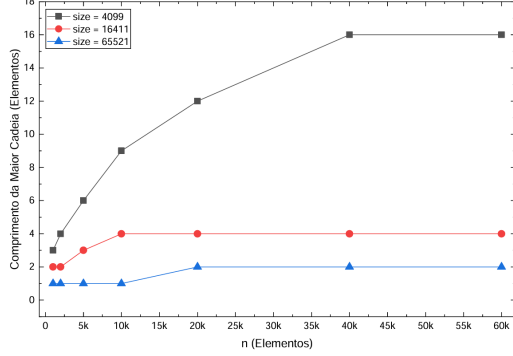


Figura 9: Comprimento da maior cadeia encadeada, evidenciando o impacto físico do subdimensionamento.

Analisando a Fig. 8 e a Fig. 9 juntas, vê-se que o volume de colisões e o comprimento da maior cadeia andam lado a lado. O gráfico de colisões mostra que a variante menor perde eficácia cedo, acumulando milhares de códigos encadeados conforme a carga aumenta. O gargalo real, porém, aparece no gráfico da maior cadeia.

As colisões começam a se acumular de forma desigual nos mesmos índices. Na tabela menor, a maior lista encadeada cresce sem parar, o que indica que, no pior caso, a ECU precisa varrer vários nós para traduzir uma única falha, degradando a performance para $O(1 + \alpha) \mid \alpha \gg 1$.

A variante de 65.521 posições mantém o total de colisões sob controle e garante que a maior cadeia fique muito curta, próxima de um único elemento.

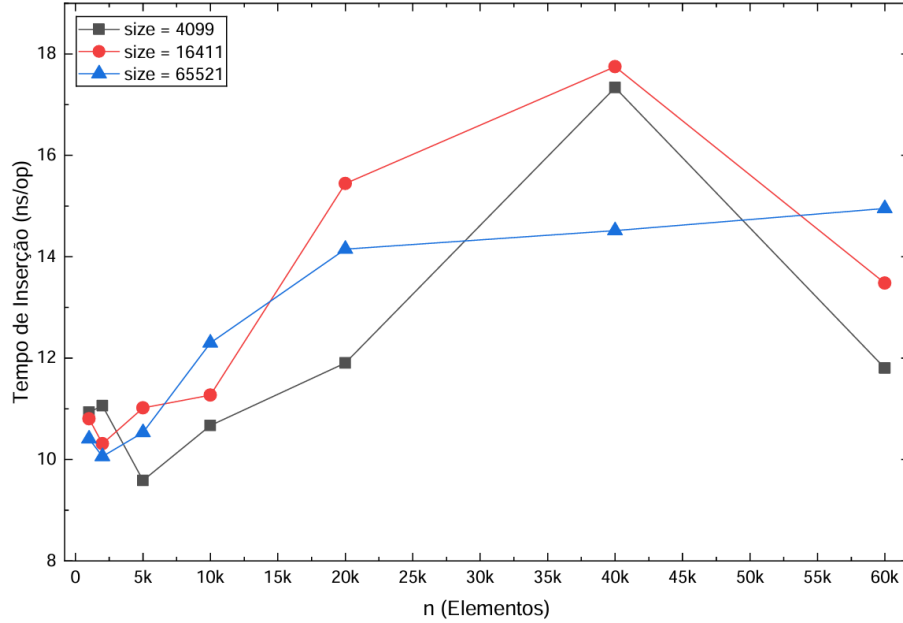


Figura 10: Estabilidade do tempo de inserção na Tabela Hash frente ao aumento do fator de carga.

Como mostra a Fig. 10, o tempo de inserção da Tabela Hash confirma a eficiência teórica $O(1)$ do tratamento por encadeamento. Como o algoritmo aloca os novos nós sempre no início da lista, ele não gasta tempo percorrendo os encadeamentos.

O resultado é um comportamento estável, com as curvas das três variantes sobrepostas na faixa de 10 a 12 nanossegundos, praticamente imunes ao aumento do fator de carga. O detalhe mais visível do gráfico é o desvio próximo a 40.000 elementos.

Esse salto para mais de 17 nanossegundos por operação não indica um colapso da estrutura, mas uma flutuação de *clock* que contaminou aquele lote de medições. A prova é que, perto dos 60.000 elementos, o tempo por operação cai de novo, o que mostra que o gargalo não veio do esgotamento da tabela.

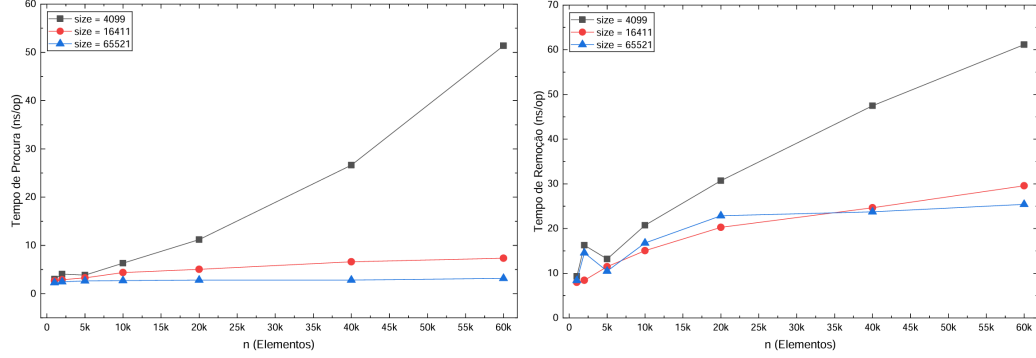


Figura 11: Degradação do tempo de busca de falhas nas tabelas com capacidade inicial insuficiente. Figura 12: Aumento do tempo de remoção causado pelo acúmulo de elementos nas mesmas listas encadeadas.

Diferente da inserção, que apenas empilha o nó no encadeamento, buscar ou remover um código de falha obriga o algoritmo a percorrer a lista daquele índice. Por isso, na Fig. 11 e na Fig. 12, o comportamento dessas duas métricas é quase um reflexo do gráfico da maior cadeia.

Na tabela subdimensionada, o tempo para encontrar ou apagar um alerta dispara conforme o volume cresce. Na marca de 60.000 elementos, a remoção, que deveria ser quase instantânea, chega perto de 60 nanossegundos. Isso mostra que o processador perde ciclos percorrendo listas longas, com a complexidade caindo para $O(1 + \alpha) \mid \alpha \gg 1$.

A variante de 65.521 posições entrega o esperado. Como o espaço amplo mantém as colisões sob controle e os encadeamentos curtos, os tempos ficam em uma linha horizontal, com complexidade $O(1 + \alpha) \mid \alpha \approx 0$.

8.3 Bloom Filter

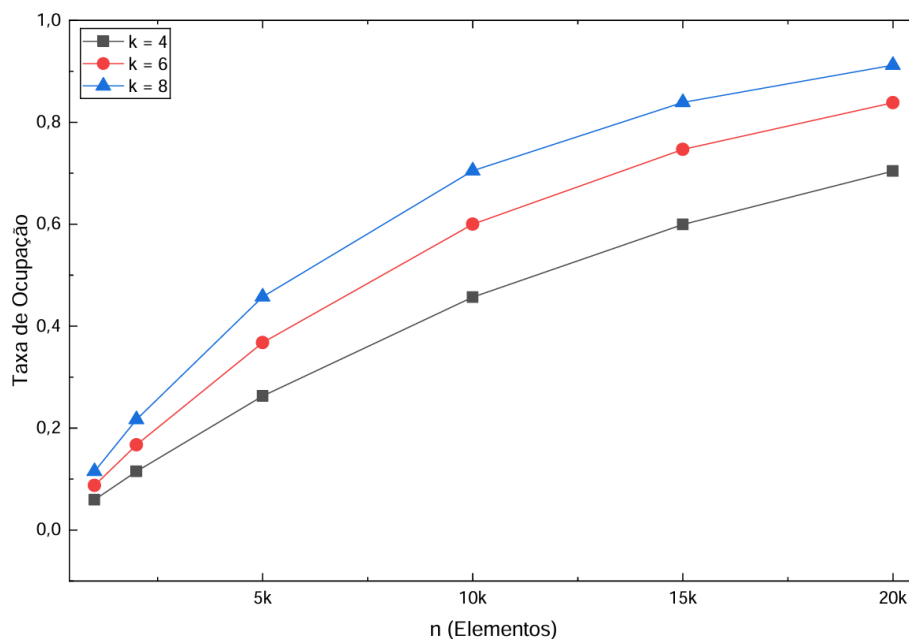


Figura 13: Saturação física do vetor de bits impulsionada pelo aumento no número de funções Hash.

A análise de memória do filtro começa na Fig. 13, que mostra o preenchimento do vetor de bits, de tamanho fixo de 65.521 bits nas três variantes, conforme a ECU processa o histórico de falhas. Como a estrutura tem tamanho estático, a taxa de ocupação reflete a proporção de bits já ligados em relação ao total.

As curvas mostram que o número de funções hash acelera esse preenchimento. A variante com 8 funções liga o dobro de bits por pacote em relação à de 4 funções e satura o espaço bem mais rápido. Ao chegar a 20.000 elementos sob estresse, essa configuração já comprometeu mais de 90% da capacidade.

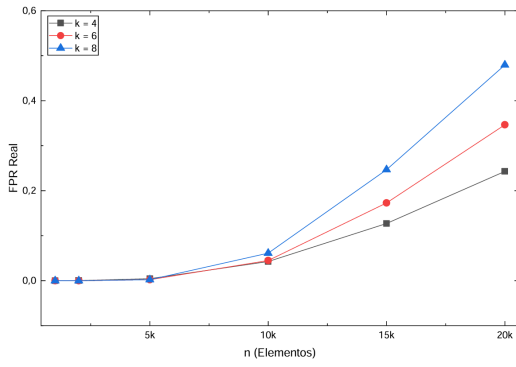


Figura 14: Fator de Falsos Positivos (FPR) real medido empiricamente durante o benchmark.

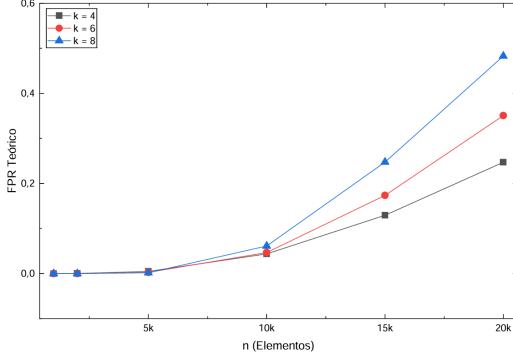


Figura 15: Curva teórica ideal do Fator de Falsos Positivos (FPR) para as configurações do filtro.

Para validar a confiabilidade do filtro, a análise compara duas visões da Taxa de Falsos Positivos (FPR), a teórica, calculada pela fórmula ideal, e a real, medida durante o benchmark por consultas de chaves ausentes que retornam falso positivo. Na Fig. 15, a teoria sugere que, com poucos elementos, um número alto de funções hash dá uma proteção quase total contra erros. Conforme a carga aumenta, porém, a Fig. 15 e a Fig. 14 mostram que essa teoria não se sustenta sob memória restrita.

Como a variante com 8 funções satura mais de 90% do vetor ao atingir 20.000 pacotes, a chance de um novo código encontrar todos os seus 8 bits já ligados por coincidência fica muito alta. O reflexo prático, na Fig. 14, é uma curva que sobe rápido e chega a 55% de falsos positivos no auge do estresse.

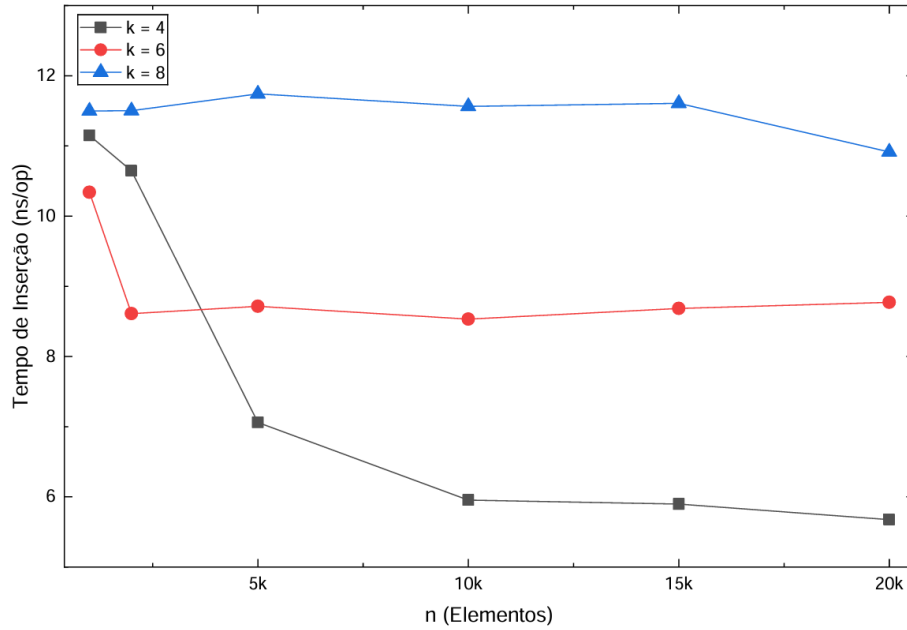


Figura 16: Custo computacional de inserção atrelado proporcionalmente à quantidade de funções Hash aplicadas.

A Fig. 16 mostra que o tempo é definido pelo número de funções hash. A variante com 8 funções faz o processador calcular oito vezes para ligar os bits, com um custo maior, na faixa de 11 a 12 nanossegundos por operação.

A configuração com 4 funções faz metade do esforço e registra tempos na casa dos 6 nanossegundos. O ponto importante é que as duas curvas ficam horizontais conforme o volume aumenta. Isso comprova que a complexidade da inserção é $O(k)$ e não depende da carga, o que garante que a entrada de novos alertas não atrasa mesmo com a estrutura cheia.

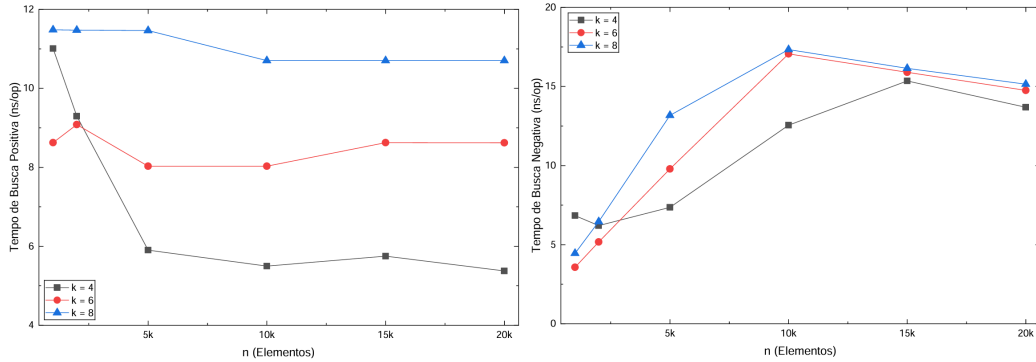


Figura 17: Tempo de busca positiva, Figura 18: Tempo de busca negativa, refletindo o processamento de todas as evidenciando o ganho de desempenho validações no vetor. da saída antecipada.

Passando da inserção para a consulta, a Fig. 17 e a Fig. 18 mostram as duas faces da verificação. Na busca positiva, quando o código de falha pertence ao conjunto, o algoritmo percorre todo o laço de verificação, checando todos os índices das funções hash para confirmar que estão ligados.

Na busca negativa, a Fig. 18 mostra a vantagem da saída antecipada. Como o filtro tem certeza de que um elemento não existe assim que encontra o primeiro bit em zero, a operação para mais cedo. Por isso, o tempo de busca negativa cresce conforme o vetor satura, pois, com mais bits ligados, o algoritmo precisa de mais saltos até achar o primeiro bit em zero, diferente da busca positiva, que tem custo constante como a inserção.

8.4 Max-Heap

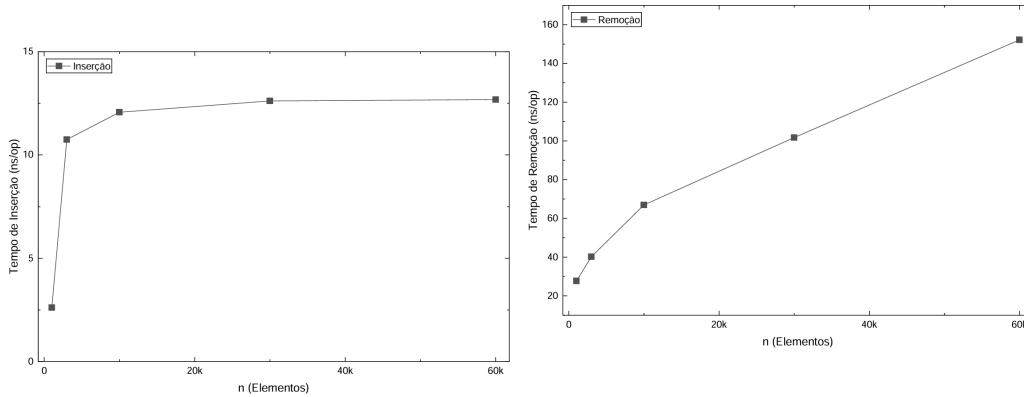


Figura 19: Estabilização logarítmica do tempo de inserção na Max-Heap.
Figura 20: Degradação do tempo de remoção da raiz devido ao acúmulo de falhas de cache (cache misses).

A Fig. 19 e a Fig. 20 mostram a diferença entre a complexidade teórica e a resposta real da memória do processador. A Fig. 19 mostra o comportamento logarítmico esperado. Depois de um declive inicial, a curva se achata e estabiliza na casa dos 12 a 13 nanossegundos por operação. Isso comprova a eficiência da subida, em que o algoritmo insere o novo pacote no fim do vetor e o promove em direção à raiz, com comparações localizadas que se resolvem rápido ao longo da pequena altura da árvore.

Na Fig. 20, porém, aparece o gargalo de hardware. Embora a remoção da raiz, que aciona a descida para reorganizar o topo, também tenha complexidade teórica $O(\log n)$, a curva real cresce de forma contínua e expressiva. O tempo parte de cerca de 30 nanossegundos por operação com a estrutura quase vazia e passa de 150 nanossegundos por operação ao chegar a 60.000 elementos.

Esse comportamento não indica erro na complexidade do algoritmo, mas sim a perda de localidade de memória. Como a árvore da Max-Heap é implícita e mapeada em um vetor único, a descida obriga o código a ler e comparar os índices dos filhos, nas posições $2i + 1$ e $2i + 2$. Para volumes baixos, o trecho usado do vetor é pequeno e cabe inteiro na cache do processador.

Conforme o vetor cresce para milhares de registros, porém, esses índices passam a apontar para posições de memória distantes umas das outras. Isso reduz a previsibilidade de leitura da CPU e gera muitas falhas de cache. O processador então perde ciclos esperando os dados virem da RAM principal. O atraso crescente que domina o gráfico de remoção é, portanto, o custo real da latência de memória, que

se soma ao previsto pela teoria.

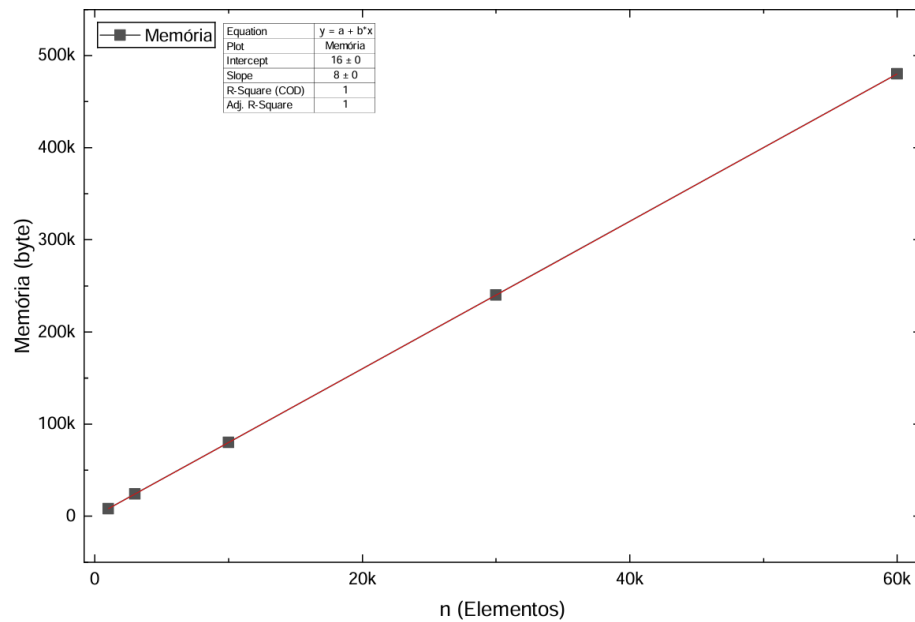


Figura 21: Crescimento linear estrito do consumo de memória na árvore implícita da Max-Heap.

A Fig. 21 mostra a principal vantagem da árvore implícita. O consumo de memória cresce de forma linear e segue a reta $y = 16 + 8x$. Esse resultado traduz a composição da estrutura no código. O coeficiente linear de 16 bytes é o custo fixo da base, que guarda apenas as variáveis de controle, como o tamanho atual, a capacidade máxima e o ponteiro para o bloco de dados.

O coeficiente angular mostra que cada elemento adicionado custa exatos 8 bytes, que correspondem ao ponteiro de referência para o alerta (DTCA`Alert*`). O registro com a carga útil do dado fica na Tabela Hash, o que evita duplicar informação pesada na árvore.

8.5 Skip List

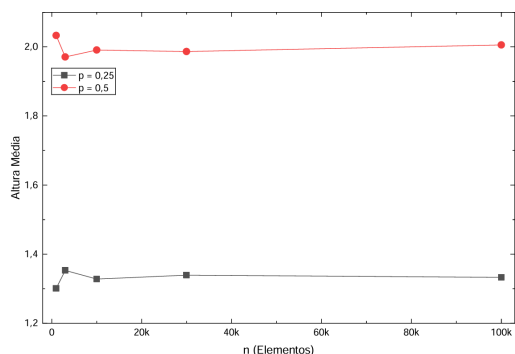


Figura 22: Influência da probabilidade de promoção na altura média dos nós da Skip List.

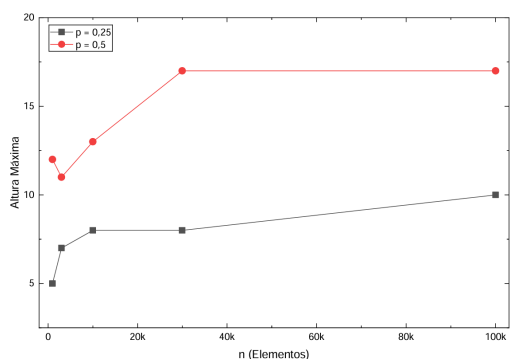


Figura 23: Crescimento da altura máxima atingida pelas torres de atalho.

A geometria da estrutura na memória aparece na análise conjunta da Fig. 23 e da Fig. 22. Os dois gráficos mostram o impacto direto da probabilidade. Com probabilidade igual a 0,5, cada nó tem 50% de chance de subir de nível, o que dá uma altura média estável em 2,0 e deixa as torres de atalho chegarem a 17 níveis com 100.000 elementos.

Com probabilidade igual a 0,25, esse crescimento em altura é bem menor, com a média na casa de 1,3 e a torre mais alta limitada a 10 níveis para a mesma carga.

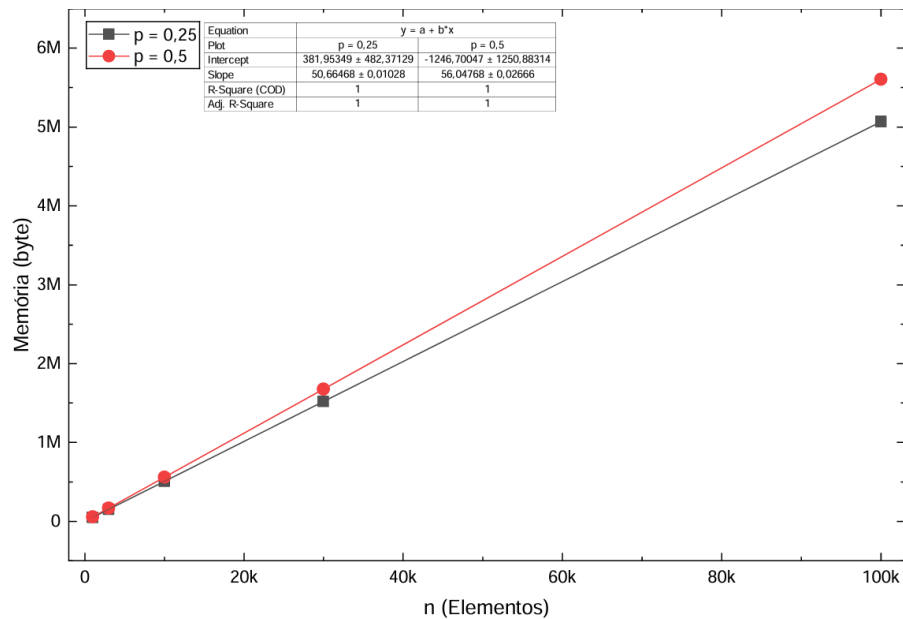


Figura 24: Gasto adicional de memória exigido para a alocação dos ponteiros de avanço.

O reflexo físico dessa arquitetura aparece na Fig. 24. Como cada nível a mais de um nó exige a alocação de um novo ponteiro de avanço, a variante com probabilidade igual a 0,5, por ter torres mais altas e mais atalhos, consome mais memória.

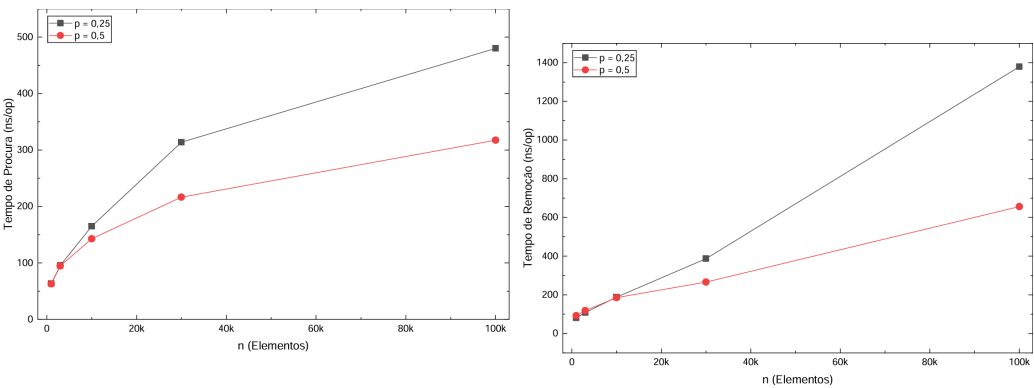


Figura 25: Redução no tempo de busca proporcionada pela abundância de rotas expressas nos níveis superiores.

Figura 26: Impacto direto do tempo de busca otimizado na velocidade da operação de exclusão de nós.

A Fig. 25 e a Fig. 26 mostram o impacto da altura nas operações de leitura. Como a variante com probabilidade igual a 0,5 tem torres mais altas, chegando a 17 níveis, o algoritmo dá saltos maiores sobre os dados e faz menos comparações para encontrar um elemento. Na Fig. 25, essa configuração estabiliza em 310 nanossegundos por operação para 100.000 elementos. A variante de probabilidade igual a 0,25, limitada a 10 níveis, exige uma travessia mais sequencial e chega a cerca de 480 nanossegundos por operação sob a mesma carga.

O mesmo comportamento se repete e se amplifica na remoção. Remover um nó exige primeiro encontrá-lo e depois atualizar os ponteiros de todos os níveis que apontavam para ele, então o custo da busca afeta direto o tempo total. Na Fig. 26, a variante com probabilidade igual a 0,25 cresce de forma contínua e chega a cerca de 1380 nanossegundos por operação no pico de estresse.

A configuração de probabilidade igual a 0,5 reduz esse atraso graças à busca mais rápida, terminando a mesma operação na faixa de 650 nanossegundos por operação. Juntos, esses resultados comprovam a troca da estrutura. O custo extra de memória e de tempo de inserção de uma probabilidade maior se justifica pelo ganho nas operações de busca e remoção.

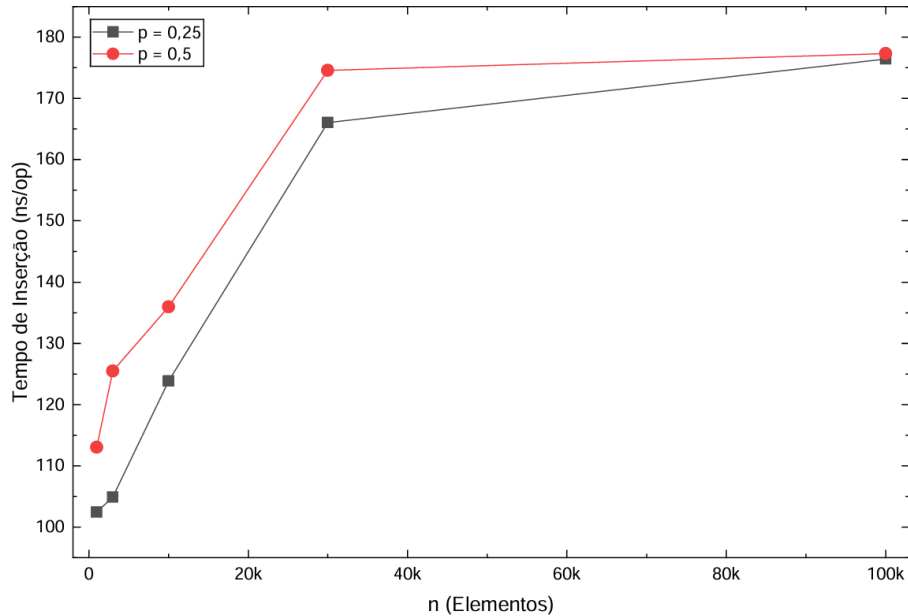


Figura 27: Custo computacional extra de inserção imposto pelo sorteio aleatório e alocação de novos níveis.

A Fig. 27 mostra o custo de inserção da Skip List conforme a carga cresce. As

duas variantes têm uma curva de crescimento acentuado que se estabiliza na faixa de 170 a 180 nanossegundos por operação ao chegar a 100.000 elementos.

A configuração com probabilidade igual a 0,5 tem tempos um pouco maiores durante a escalada da estrutura. Isso acontece porque, com mais promoções, o algoritmo chama mais vezes o gerador de números aleatórios, faz mais alocações de nós por elemento e atualiza mais ponteiros de travessia em cada nível alcançado.

9 Conclusão

O desenvolvimento deste sistema de telemetria mostrou que a escolha de algoritmos para sistemas embarcados vai além da teoria e esbarra nos limites reais do hardware. Com dados sintéticos baseados no motor Volkswagen AP 1.6 8V turbo, a arquitetura foi testada sob estresse, ruído e injeção de falhas, o que validou o fluxo de processamento da ECU e revelou os gargalos de desempenho da máquina.

Os testes isolados confirmaram o impacto das operações de memória. No buffer de recepção, a Fila Estática Circular se mostrou essencial para receber dados rápidos, evitando a lentidão da alocação dinâmica. A Tabela Hash foi rápida, mas mostrou que precisa de espaço de sobra para não atrasar as buscas. O Bloom Filter economizou memória ao barrar alertas repetidos, embora sua precisão caia rápido conforme o vetor satura.

Na gestão de prioridades, a Max-Heap ordenou os alertas por gravidade de forma correta, mas perdeu velocidade com volumes grandes por causa das falhas de cache ao percorrer o vetor. A Skip List registrou o histórico e mostrou que o gasto extra de memória com ponteiros compensa, pois agiliza as buscas por tempo sem exigir o rebalanceamento contínuo de uma árvore.

Por fim, o sistema integrado funcionou de forma estável com as cinco restrições aplicadas, rodando em núcleo único e travado no limite de 128 MB de RAM, o que obrigou a exportação gráfica a sair da rotina principal. Fica claro que projetar software de tempo real exige equilibrar perdas e ganhos. Em sistemas embarcados, muitas vezes se cede espaço de memória para ganhar velocidade, garantindo que o módulo de diagnóstico aponte a falha a tempo de salvar a integridade mecânica do veículo.

10 Apêndice

<https://claude.ai/share/8ebf8648-63e2-495e-9768-d12845d95fcc>

<https://claude.ai/share/97e8f61d-b35f-4ee4-9bdb-612d05ef56fc>

<https://claude.ai/share/a86398db-6bfc-4668-998d-54a74debbf10>

<https://claude.ai/share/29c9adfa-686b-45bd-8c61-56895f59f92f>

11 Bibliografia

Referências